

This is analogous to evolution issue 98. [EWG98](#) This paper also resolves core issue 1617. [CWG1617](#)

Robert Haberlach (rh633{at}cam{dot}ac.uk)

2016-03-28

Motivation

The `alignas` specifier is applicable to objects, affecting their alignment requirement but not their type. It is therefore currently not possible to determine an object's actual alignment requirement. This paper proposes to permit application of `alignof` to objects and references.

Proposed wording

Modify 3.11 [basic.align] ¶2 as indicated:

When applied to a type, the result of the `alignof` operator reflects the alignment requirement **of the type** in the complete-object case.

Augment 5 [expr] ¶8 as indicated:

In some contexts, unevaluated operands appear (5.2.8, 5.3.3, **5.3.6**, 5.3.7, 7.1.6.2).

Augment 5.3 [expr.unary] ¶1 as indicated:

unary-expression:
 `alignof ( type-id )`
 `alignof postfix-expression`

Modify 5.3.6 [expr.alignof] as indicated:

1. An `alignof` expression yields the alignment requirement of its operand **type**. **If the operand is a type, it shall represent** ~~representing~~ a complete object type, or an array thereof, or a reference to one of those types. **Otherwise, the operand shall be a possibly parenthesized postfix-expression that is an id-expression or class member access (5.2.5 [expr.ref]). The postfix-expression shall designate a variable or class data member. The expression is an unevaluated operand (clause 5 [expr]).**
2. The result is an integral constant of type `std::size_t`.
3. When `alignof` is applied to a reference type, the result is the alignment of the referenced type. When `alignof` is applied to an array type, the result is the alignment of the element type.
4. **When the operand is a postfix-expression e, the result is determined as follows:**
 - **If e designates an entity of object type, and any preceding declarations of that entity have an alignment-specifier (7.6.2 [dcl.align]), the result is the alignment specified by those declarations. [Note: All declarations of an entity that have an alignment-specifier must specify equivalent alignment; see 7.6.2 [dcl.align]. —end note]**
 - **Otherwise, the result is alignof(decltype(e)).****[Note: As is the case with noexcept, subsequent applications of alignof to the same entity may yield different values. — end note]**

Modify 7.6.2 [dcl.align] ¶6 as indicated (this resolves core issue 1617 [CWG1617](#)):

If **the defining any** declaration of an entity has an *alignment-specifier*, **any non-defining all** declarations of that entity shall either specify equivalent alignment or have no *alignment-specifier*. ~~Conversely, if any declaration of an entity has an alignment-specifier, every defining declaration of that entity shall specify an equivalent alignment.~~ No diagnostic is required if declarations of an entity have different *alignment-specifiers* in different translation units. [Example: ... —end example] **[Example:**

```
alignas(16) extern int i;  
  
alignas(8) extern int i;           // error: Different alignment specified  
  
extern int i;                     // Ok: No alignment-specifiers
```

—end example]

Augment 14.6.2.2 [temp.dep.expr] ¶4 as indicated:

Expressions of the following forms are never type-dependent (because the type of the expression cannot be dependent):

[...]

`alignof ( type-id )`

`alignof postfix-expression`

[...]

Augment 14.6.2.3 [temp.dep.constexpr] ¶4 as indicated:

Expressions of the following form are value-dependent:

[...]

`alignof postfix-expression`

Acknowledgments

The author would like to thank Daveed Vandevoorde for assistance in preparing the wording.

References

[EWG98] “[tiny] Core issue 1008, Querying the alignment of an object“: wg21.link/ewg98

[CWG1617] “alignas and non-defining declarations“: wg21.link/1617